
Part B

Ex: 12

Title: Delivery Route Optimization.

Problem Description: Imagine a delivery company that operates in a city with five key delivery locations. The company needs to design a route for their delivery vehicle that visits each location exactly once and returns to the starting point without retracing its steps.

Method: Given a graph representing the locations as vertices and the roads between them as edges, determine whether a Hamiltonian Circuit exists. If it does, find the circuit.

Theory Reference: Module 4

Algorithm

Step 1: Initialize a temporary array `temp` to store the current path during recursion.

Step 2: Check base case:

- If all nodes are visited (`visited == n`) and there is an edge from the current node (`st`) back to the source, then:
 - A Hamiltonian circuit is found.
 - Call the function `path(succ)` to process or print this path.
 - Return from the function.

Step 3: Recursive step:

- For every vertex `i` from 0 to `n-1`:
 - If there is an edge from the current node `st` to `i` (`a[st][i] == 1`) and `i` has not been visited yet (`succ[i] == 0`):
 - Assign `i` as the successor of `st` in the path: `succ[st] = i`
 - Copy the `succ` array to `temp` to preserve path state
 - Recursively call `ckt(i, temp, visited + 1)` to continue the path from node `i`

Program

```
import java.util.Scanner;

public class Hamiltonian
{
    static int a[][],n,source;
```

```
static void path(int succ[])
{
    System.out.print(source+"->");
    for(int j=succ[source];j!=0;j=succ[j])
        System.out.print(j + "->");
    System.out.println(source);
}

static void ckt(int st,int succ[],int visited)
{
    int[] temp=new int[n];

    //Check if all the nodes are visited and there is path from the last node to first node
    if(visited==n && a[st][source]==1)
    {
        path(succ);
        return;
    }

    //Build hamiltonian circuit by considering path from every neighbour node of source
    for(int i=0;i<n;i++) {
        if(a[st][i]==1 && succ[i]==0)
        {
            succ[st]=i;

            //Create temporary visited path so far in temp array
```

```
        System.arraycopy(succ, 0, temp, 0, n);

        ckt(i,temp,visited+1);

    }

}

}

public static void main(String[] args)

{

    Scanner s = new Scanner(System.in);

    System.out.println("Enter the no. of delivery locations in a city/vertices");

    n = s.nextInt();

    int[] succ=new int[n];

    a = new int[n][n];

    System.out.println("Enter the roads between delivery locations/adjacency matrix");

    for(int i=0;i<n;i++){

        for(int j=0;j<n;j++){

            a[i][j]=s.nextInt();

        }

    }

    System.out.println("Routes for delivery vehicle/Hamiltonian circuits");

    ckt(source=0,succ,1); //Build Hamiltonian Circuit from the node 0

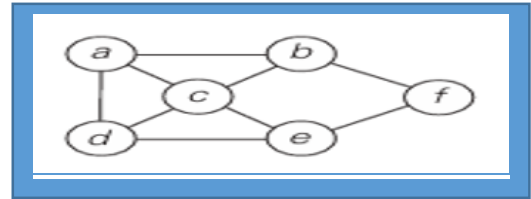
}

}
```

Output

Enter the no. of delivery locations in a city/vertices

6



Enter the roads between delivery locations/adjacency matrix

0 1 1 1 0 0

1 0 1 0 0 1

1 1 0 1 1 0

1 0 1 0 1 0

0 0 1 1 0 1

0 1 0 0 1 0

Routes for delivery vehicle/Hamiltonian circuits

1->2->6->5->3->4->1

1->2->6->5->4->3->1

1->3->2->6->5->4->1

1->3->4->5->6->2->1

1->4->3->5->6->2->1

1->4->5->6->2->3->1

BUILD SUCCESSFUL (total time: 13 seconds)

```
run:
Enter the no. of delivery locations in a city/vertices
6
Enter the roads between delivery locations/adjacency matrix
0 1 1 1 0 0
1 0 1 0 0 1
1 1 0 1 1 0
1 0 1 0 1 0
0 0 1 1 0 1
0 1 0 0 1 0

Routes for delivery vehicle/Hamiltonian circuits
1->2->6->5->3->4->1
1->2->6->5->4->3->1
1->3->2->6->5->4->1
1->3->4->5->6->2->1
1->4->3->5->6->2->1
1->4->5->6->2->3->1
BUILD SUCCESSFUL (total time: 13 seconds)
```